

Elevator RF Survey App - Implementation Specification

Status: Build brief / single source of truth for implementation

Audience: Claude Code and the delivery team

Scope note: Statements labelled **Existing/confirmed** come from the supplied *Elevator RF Survey App Technical Report*. Statements labelled **Proposed** are deliberate additions required for the requested shared web dashboard, backend, sync, and future cloud deployment. Do not treat a proposed item as existing behavior.

1. Product goal

Build the next version of the Elevator RF Survey App: an Android-first field-survey application and searchable web dashboard for elevator RF/LTE feasibility visits.

An engineer must be able to complete the standard Site Visit Sheet at the site, including elevator assessment, technical findings, railing measurements/drawings, RF readings, pinout, checklist, and photos; generate the approved-format Word report; and later find the survey in a dashboard by site, elevator, engineer, status, date, carrier, or feasibility result.

The first test release is hosted from the user's laptop. MongoDB Atlas Free stores structured survey data; photos and generated files are stored separately. The architecture must make later cloud hosting a configuration/deployment change, not a rewrite.

In scope

- Preserve the current Site Visit Sheet terminology, sections, calculations, DOCX-template approach, and Android-native behavior where confirmed below.
- Add authenticated multi-user storage, dashboard search/filter/detail views, synchronization, audit fields, and server-managed files.
- Support Android (Capacitor) and web from a shared frontend/domain model.
- Preserve offline survey work and make sync explicit, resumable, and conflict-safe.

Out of scope for the MVP/test release

- Replacing the client-approved DOCX layout with a custom report renderer.
- App-generated exact-format PDFs (Word/Docs remains the PDF renderer).
- iOS native modem reads.
- Public/anonymous access to surveys.
- Full enterprise multi-tenancy, SSO, real-time collaboration, scheduled reports, analytics warehouse, AI image analysis, or electronic signatures. These may be proposed later, not silently added.

2. Existing product - preserve these behaviors

2.1 Current architecture (confirmed)

The current application is one HTML/CSS/JavaScript source, served as a web page and wrapped in Capacitor for Android. It auto-saves one complete draft JSON object in `localStorage` under `svs_draft_v3`, with a debounce on edits; reload restores the draft. It has no server or database.

The existing repo contains the live `template.docx`, bundled PizZip and docxtemplater libraries, Android plugin sources, a template-building Python tool, and a GitHub Actions Android build. Do not discard these simply because the new backend is Python.

2.2 Existing Site Visit Sheet content and vocabulary (confirmed)

Keep these names in UI labels, API names where practical, and report mapping:

1. **Header:** Document Ref, Prepared By, Date, Version, Project, Site Name, Client, Address, Maps link.
2. **1.1 Elevator Assessment:** 13-item Yes / No / N-A checklist and free-text note.
3. **2. Technical Findings - Elevator:** OEM, Model, Elevator ID, Maintenance Company, Plug points.
4. **Railing measurements:** Top, Rear, Left, Right; lettered dimensions A, B, C... captured through the drawing board.
5. **3. LTE / RF Survey:** RSRP, RSRQ, SINR, Band, DL, UL for DU and e& in charging area + lobby, on top of cabin by floor, and inside cabin by floor. Includes automatic averages and feasibility.
6. **4. Elevator Integration Checks:** button module SKU, connectors, four-pin USE/Color pinout.
7. **5. Photos & Attachments:** description and filename, one page per photo in the DOCX.

2.3 Existing draft shape that must be mapped, not guessed (confirmed)

```
svs_draft_v3
  fields: doc_ref, prepared_by, date, version, project, site_name,
          client, address, maps_link, elev_oem, elev_model, elev_id,
          elev_maint, plug_points, checklist_note, networks,
          survey_point, feasibility, btn_sku, btn_connectors, rail_note, ...
  checklist: { "0": "Yes" | "No" | "N/A", ... } // 13 items
  rails: { top|rear|left|right: { dims: { A, B, C, ... }, draw: [shape, ...] } }
  pinout: [{ use, color }, ...] // 4 pins
  rf: { rf1: { "<row>|<carrier>.<metric>": value }, rf2: {...}, rf3: {...} }
  photos: [{ desc, filename, img: "data:image/jpeg;base64,..." }]
  floorCount: "25"
  carrier: "du" | "et"
```

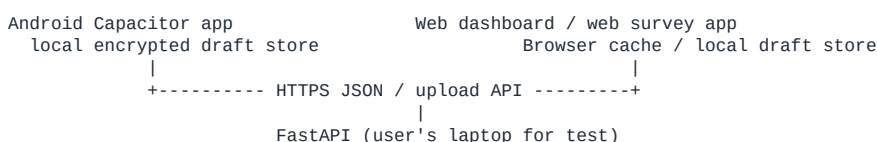
The flat RF key format is significant: `3|du.rsrp` means floor-table row 3, DU carrier, RSRP metric. Both carrier values must remain available even when the UI toggles which carrier is shown.

Each railing's draw is a 340 x 240 vector scene holding `rect`, `line`, `arrow`, `path`, `dim`, `meas`, and `text` shapes. Labels and measurements are only exported when actually measured. Preserve raw drawing data.

2.4 Existing RF and DOCX behavior (confirmed)

- The feasibility rule is: **Good** when average RSRP ≥ -90 and SINR ≥ 12 ; **Acceptable** when RSRP ≥ -105 and SINR ≥ 5 ; otherwise **Poor**. It is calculated across populated readings and may be manually overridden in the current form.
- DOCX is generated offline on the device using the existing `template.docx`, PizZip, and docxtemplater. Preserve the approved template's formatting and loop tags for RF rows, photos, and railing data.
- Android saves/opens the exported DOCX using `NativeFile`; web uses a Blob download.
- Android sends the DOCX silently to Telegram on export and also has a manual **Send to Telegram** action through a native multipart plugin. Browser Telegram delivery is not supported because of CORS.
- Native Android can read cell signal values and can use Cloudflare speed endpoints. Web users type modem values manually. Keep Android-only capabilities conditional; never promise them in web/iOS.

3. Target architecture





3.1 Components

Component	MVP/test responsibility	Production evolution
Frontend	Keep/refactor existing shared web app; Capacitor Android wrapper; add dashboard routes	Serve static web app from CDN/hosting; release signed mobile builds
FastAPI	Auth, CRUD, sync, search, file upload/download authorization, dashboard API, audit writes	Containerized/redundant service behind TLS reverse proxy
MongoDB Atlas	Atlas Free database for users, sites, elevators, survey documents, audit/events, report metadata	Paid/appropriately sized cluster, monitoring, point-in-time recovery where available
File storage	Local laptop directory outside the web root for test photos/DOCX files	S3-compatible private bucket (or approved managed object storage)
DOCX	Continue offline client generation using the approved template; upload the exported copy/metadata when online	Proposed: optional server-side regeneration only after template/version governance is agreed

3.2 Architectural rules

1. Do not store Base64 image bodies or DOCX bytes inside MongoDB survey documents. MongoDB stores file metadata and an opaque storage key.
2. The frontend never receives MongoDB credentials, Telegram bot credentials, or storage credentials.
3. Use a storage adapter interface: `LocalStorage` for laptop testing and `S3FileStorage` later. Both expose the same methods: `put`, `open/read stream`, `delete` (admin-only retention job), and `signed/read-authorized URL generation`.
4. Keep a versioned domain schema and a migration/import adapter for `svs_draft_v3` so existing drafts can be imported without loss.
5. All API dates are ISO 8601 UTC. Display in the user's local time zone. Measurements retain their original units (mm). RF values retain their original reported units.

4. Data model and MongoDB collections

Use separate collections for independent identity/search (sites, elevators, users) and a self-contained surveys document for the variable, report-shaped data. This avoids forcing variable RF rows and drawings into artificial relational tables.

All primary documents include:

```

{
  "_id": "ObjectId",
  "schema_version": 1,
  "created_at": "2026-09-11T00:00:00Z",
  "created_by": {"user_id": "...", "name": "..."},
  "updated_at": "2026-09-11T00:00:00Z",
  "updated_by": {"user_id": "...", "name": "..."},
  "revision": 1,
  "is_deleted": false
}

```

`revision` is mandatory and increments on each accepted mutation. It supports offline conflict detection. Do not physically delete test records from normal application flows; use `is_deleted/status` and retain audit information.

4.1 users

```
{
  "email": "engineer@example.com",
  "display_name": "Engineer Name",
  "role": "engineer",
  "password_hash": "...",
  "is_active": true,
  "last_login_at": "...",
  "created_at": "...",
  "updated_at": "..."
}
```

Roles: admin, manager, engineer, viewer. See authorization rules below. password_hash must be Argon2id or bcrypt, never plaintext.

4.2 sites

```
{
  "site_code": "optional-unique-human-code",
  "name": "Site Name",
  "client": "Client",
  "address": "Address",
  "maps_link": "https://...",
  "status": "active",
  "search_text": "normalized server-maintained search string",
  "created_at": "...", "created_by": {"user_id": "...", "name": "..."},
  "updated_at": "...", "updated_by": {"user_id": "...", "name": "..."}, "revision": 1
}
```

4.3 elevators

```
{
  "site_id": "ObjectId",
  "elevator_id": "existing elev_id value; not globally assumed unique",
  "oem": "existing elev_oem value",
  "model": "existing elev_model value",
  "maintenance_company": "existing elev_maint value",
  "plug_points": "existing plug_points value",
  "status": "active",
  "search_text": "...",
  "created_at": "...", "created_by": {"user_id": "...", "name": "..."},
  "updated_at": "...", "updated_by": {"user_id": "...", "name": "..."}, "revision": 1
}
```

4.4 surveys - canonical report-shaped record

```
{
  "survey_number": "EV-2026-001",
  "status": "draft | submitted | reviewed | exported | archived",
  "site_id": "ObjectId",
  "elevator_id": "ObjectId",
  "header": {
    "doc_ref": "", "prepared_by": "", "date": "YYYY-MM-DD", "version": "",
    "project": "", "site_name_snapshot": "", "client_snapshot": "",
    "address_snapshot": "", "maps_link_snapshot": ""
  },
  "elevator": {
    "oem": "", "model": "", "elev_id": "", "maintenance_company": "", "plug_points": ""
  },
  "assessment": {"checklist": {"0": "Yes"}, "note": ""},
  "railings": {
    "top": {"dimensions_mm": {"A": 123}, "drawing": [{"type": "line", "...": "preserved shape"}]},
    "rear": {"dimensions_mm": {}, "drawing": []},
    "left": {"dimensions_mm": {}, "drawing": []},
    "right": {"dimensions_mm": {}, "drawing": []},
    "note": ""
  },
  "rf": {
    "floor_count": 25,
    "active_carrier": "du",
    "sections": {
      "rf1": {"name": "Charging area + lobby", "cells": {"0|du.rsrp": -90}},
      "rf2": {"name": "On top of cabin", "cells": {"3|du.rsrp": -95}},
      "rf3": {"name": "Inside cabin", "cells": {"3|du.rsrp": -102}}
    },
    "summary": {
      "computed_at": "...",
      "overall": {"averages": {"rsrp": -98, "sinr": 8}, "verdict": "Acceptable"},
      "by_carrier": {"du": {"averages": {}, "verdict": ""}, "et": {"averages": {}, "verdict": ""}},
      "recommended_operator": "du"
    }
  },
}
```

```

    "feasibility": {"value": "Acceptable", "source": "computed | manual_override", "override_note": ""}
  },
  "integration": {"button_module_sku": "", "connectors": "", "pinout": [{"pin": 1, "use": "", "color":
""}]},
  "photos": [{"photo_id": "ObjectId", "description": "", "filename": "", "captured_at": "...",
"sort_order": 1}],
  "reports": [{"report_id": "ObjectId", "file_id": "ObjectId", "template_version": "", "format": "docx",
"created_at": "...", "created_by": {"user_id": "...", "name": "..."}, "telegram": {"attempted_at": "...",
"result": "sent | failed | not_attempted", "message": "safe error summary"}},
  "legacy": {"source": "svs_draft_v3", "imported_at": "...", "original_draft_version": "v3"},
  "created_at": "...", "created_by": {"user_id": "...", "name": "..."},
  "updated_at": "...", "updated_by": {"user_id": "...", "name": "..."}, "revision": 1, "is_deleted":
false
}

```

Important mapping choices:

- Retain RF cells in the confirmed flat-key form to ensure lossless migration and carrier-toggle compatibility. The API may additionally return normalized display rows; it must not overwrite the source cells.
- Store the survey's site/elevator display values as snapshots so historical reports do not change when the site/elevator directory is edited later.
- `dimensions_mm` stores only explicitly measured values. Do not calculate missing letters from drawing pixels as a new rule.
- The client/server should recompute `rf.summary` whenever RF cells change. Preserve a manual feasibility override with source and reason; never silently replace it.

4.5 files

```

{
  "owner_type": "survey",
  "owner_id": "ObjectId",
  "kind": "photo | report_docx | attachment",
  "storage_key": "surveys/<survey-id>/photos/<uuid>.jpg",
  "original_filename": "cabin.jpg",
  "content_type": "image/jpeg",
  "byte_size": 123456,
  "sha256": "...",
  "image": {"width": 1280, "height": 960, "quality_profile": "test-low"},
  "created_at": "...", "created_by": {"user_id": "...", "name": "..."},
  "is_deleted": false
}

```

4.6 audit_events

Append-only business audit log, not a replacement for infrastructure logs:

```

{"event_type": "survey.updated", "entity_type": "survey", "entity_id": "...", "revision": 4,
"actor": {"user_id": "...", "name": "..."}, "at": "...", "request_id": "...",
"summary": {"changed_paths": [{"rf.sections.rf2.cells"}]}}

```

Do not store passwords, bearer tokens, photo bodies, or raw personal data in audit events.

4.7 Indexes

- `users.email`: unique.
- `sites.site_code`: unique sparse; `sites.name`, `sites.client`, `sites.search_text`.
- `elevators`: compound unique sparse on `{site_id, elevator_id}`; indexes on `site_id`, OEM/model search fields.
- `surveys.survey_number`: unique; indexes on `{site_id, updated_at}`, `{elevator_id, updated_at}`, `{status, updated_at}`, `{created_by.user_id, updated_at}`, `rf.feasibility.value`, and a text/search index over survey number plus header/site/elevator snapshots.
- `files.owner_id`, `audit_events.{entity_id, at}`.

Start with these predictable indexes; measure before adding more.

5. API contract

Use `/api/v1`, JSON request/response bodies, Pydantic models, RFC 7807-style error objects, request IDs, and OpenAPI documentation. Require authentication except health/readiness endpoints. Object IDs are serialized as strings.

5.1 Authentication and session endpoints

Method	Path	Purpose
POST	<code>/auth/login</code>	Email/password login; returns short-lived access token and refresh token/session per client platform
POST	<code>/auth/refresh</code>	Rotate refresh token and issue new access token
POST	<code>/auth/logout</code>	Revoke the current refresh session
GET	<code>/auth/me</code>	Current user, role, permissions
POST	<code>/users</code>	Admin creates a user
GET/PATCH	<code>/users/{id}</code>	Admin manages user/role/active state; users may update only their own safe profile fields if implemented

For the MVP, local email/password accounts are sufficient. **Proposed production addition:** OIDC/SSO after an identity provider is selected.

5.2 Directory and survey endpoints

Method	Path	Purpose
GET/POST	<code>/sites</code>	Paginated list/search; create site
GET/PATCH	<code>/sites/{id}</code>	Read/update site
GET/POST	<code>/sites/{site_id}/elevators</code>	List/create elevator under a site
GET/PATCH	<code>/elevators/{id}</code>	Read/update elevator
GET/POST	<code>/surveys</code>	Dashboard list with filters; create a server draft
GET	<code>/surveys/{id}</code>	Full survey document, current revision, file metadata
PATCH	<code>/surveys/{id}</code>	Partial update with required If-Match revision or base_revision
POST	<code>/surveys/{id}/transition</code>	Explicit status transition (draft -> submitted, etc.)
POST	<code>/surveys/{id}/recalculate-rf</code>	Recompute RF summary/feasibility; invoked by client or update handler
POST	<code>/surveys/{id}/import-legacy-draft</code>	Admin/owner import of validated svcs_draft_v3 payload
GET	<code>/surveys/{id}/audit</code>	Authorized audit timeline
POST	<code>/sync/push</code>	Batch offline mutations with operation UUID and base revision
GET	<code>/sync/pull?since=<cursor></code>	User-authorized changes since cursor

GET `/surveys` query parameters: `q`, `site_id`, `elevator_id`, `client`, `status`, `feasibility`, `carrier`, `engineer_id`, `date_from`, `date_to`, `has_photos`, `sort`, `page`, and `page_size`. Whitelist values; do not pass arbitrary Mongo queries from the client.

5.3 Files and reports

Method	Path	Purpose
POST	/surveys/{id}/photos	Multipart upload (or initiate direct-upload flow later); creates file metadata and ordered photo reference
GET	/files/{id}	Authorized streamed/signed download; content disposition appropriate to file type
PATCH	/surveys/{id}/photos/{photo_id}	Change description/order only
DELETE	/surveys/{id}/photos/{photo_id}	Soft-delete reference/file metadata under authorized retention rules
POST	/surveys/{id}/reports	Register uploaded client-generated DOCX and template version
GET	/surveys/{id}/reports	Report history
GET	/surveys/{id}/reports/{report_id}/download	Authorized DOCX download
POST	/surveys/{id}/telegram/send	Proposed server-side option, disabled by default in MVP ; only if secure Telegram credential ownership is agreed

For MVP, preserve the established Android native Telegram plugin; record the delivery result when the app syncs the export event. Do not move the bot token into browser JavaScript or MongoDB.

5.4 Dashboard endpoint

GET /dashboard/summary returns only aggregate counts/statistics scoped to the caller's authorization: total results, draft/submitted/exported counts, Good/Acceptable/Poor totals, selected date range, and most recent updates. It must use the same filters as the list endpoint to avoid misleading counts.

6. Authentication and authorization

Role	Access
admin	Manage users/configuration, all sites/elevators/surveys/files/audit, archive records, restore backups through operational process
manager	Read all company surveys, create/edit/review/export where business policy permits, view audit/report history; cannot manage admins/secrets
engineer	Create surveys; read/edit own drafts and assigned surveys; upload photos; export own/assigned surveys; read permitted directories
viewer	Read-only dashboard/detail/DOCX download for permitted records; no photo/report mutation

For test deployment, scope all logged-in users to one company. **Proposed production feature:** organization/tenant IDs and per-site assignment policy if multiple customers must be separated. Enforce authorization in FastAPI service code for every object lookup, not merely by hiding frontend buttons.

Use HTTPS even on the laptop when accessed by phones beyond localhost; use a trusted reverse proxy/tunnel or a local network certificate approach appropriate to the test environment. Access tokens are short-lived. Store web refresh tokens in secure, HTTP-only, same-site cookies where the deployment topology allows; mobile uses secure device storage. Rate-limit login, validate passwords, revoke sessions after disable/password reset, and log security-relevant events.

7. Photo and attachment strategy

MVP/test profile

- Preserve camera/gallery acquisition. Generate JPEG before upload.
- Default test profile: longest edge **1280 px**, JPEG quality **0.65**. Target ≤ 500 KB per photo where image content permits; do not reject a photo solely because it cannot meet the target after normal compression.
- Strip EXIF metadata unless a confirmed business need requires it; record capture time only when explicitly available/approved.
- Allow only JPEG/PNG/WebP input initially; re-encode accepted images to JPEG. Reject executable/unknown MIME types, enforce maximum upload size (for example 8 MB pre-transform), use UUID storage filenames, and never trust the client filename/path.
- Keep description, filename, sort order, dimensions, checksum, and test quality profile in metadata.
- On laptop, save files under a configured private data directory outside the frontend/static directory. Serve only through authenticated API routes.

Production profile (proposed)

Move the exact same storage-key contract to a private S3-compatible bucket. Use server-authorized, short-lived upload/download URLs or proxy streams. Add malware scanning/quarantine if company policy requires it. Define image retention/deletion rules with the business before enabling automatic cleanup.

8. Offline-first drafts and sync

Offline support is mandatory because the confirmed current app works locally and elevator sites may have weak signal.

Local behavior

1. Retain `svs_draft_v3` read support for migration. New app data uses a versioned local store (IndexedDB for web; Capacitor-compatible local database/storage for Android) rather than relying only on one `localStorage` key.
2. Save every edit locally first. Show clear state: `Saved on device`, `Syncing`, `Synced`, `Offline - changes waiting`, or `Conflict needs review`.
3. Store photos locally until upload succeeds. Queue binary upload separately from the survey JSON mutation; retry with backoff when online.
4. Assign a client UUID to every pending operation. The server stores/recognizes it so retrying after a timeout is idempotent.
5. Export DOCX must remain possible from a complete local draft while offline.

Sync protocol and conflicts

- A queued mutation contains `operation_id`, `survey_id`, `base_revision`, changed paths/payload, client timestamp, and local draft schema version.
- Server accepts only if `base_revision` is current, then returns the new revision. On a mismatch return HTTP 409 plus the current server document or a conflict envelope.
- Auto-merge non-overlapping paths only when deterministic and audited. For overlapping scalar fields, never silently use last-write-wins. Present a simple choice: keep device value, keep server value, or merge manually.
- RF cells and photo operations can often merge by key/ID when distinct; identical RF cell keys conflict. Railing drawings are treated as one railing object for conflict purposes unless a safe shape-level merge is explicitly designed.
- A survey is not marked submitted until all required data validation passes and all required queued photo uploads either complete or the user deliberately removes the reference.

Proposed enhancement after MVP: background sync and push notifications. Do not make them a prerequisite for reliable manual sync.

9. Dashboard and web user experience

Build the dashboard to match the requested concept: an office-friendly, searchable survey register with a left navigation/filter area, statistic cards, a main survey table, and a focused survey-detail workspace. Do not invent visual elements as if they were present in the source report; use this as the implementation definition for the requested dashboard.

9.1 Dashboard landing page

- Header: product name, global search, current user/profile, and New survey.
- Search: survey number/document ref, site, client, address, elevator ID, OEM, model, prepared-by text. Debounced server search; preserve selected filters in URL.
- Filter controls: status, feasibility, carrier data present, client, site, engineer, date range, photos present. Include Clear filters and display active filter count.
- Statistic cards: total matching surveys, Drafts, Submitted/Reviewed, Exported, and feasibility distribution (Good / Acceptable / Poor). Counts reflect filters.
- Survey table: Survey/document reference, site, elevator ID/OEM, client, survey date, prepared by, feasibility badge, status, last updated, photo/report indicators, and row action. Default sort: most recently updated.
- Pagination/server-side sorting; no loading of all records to the browser. Empty, loading, error, and no-results states are required.

9.2 Survey detail page/tabs

Show immutable identity/status summary at top, including sync/revision state. Tabs:

1. **Overview:** header, site/elevator snapshots, assessment checklist, technical findings, feasibility, owner/status, audit summary.
2. **RF Summary:** overall and per-carrier averages, verdict/recommendation, separate section summaries for charging area + lobby, top of cabin, inside cabin; expandable raw grid retaining original flat keys/values.
3. **Railing Measurements:** Top/Rear/Left/Right dimensions, notes, and read-only rendering of the saved vector drawing (with raw source retained). Clearly distinguish no measurement from zero.
4. **Pinout & Integration:** button module SKU, connectors, exactly four USE/Color entries unless the existing form later confirms another cardinality.
5. **Photos:** thumbnail grid, descriptions, filenames, capture metadata; authenticated full-size open/download; upload/reorder/edit description for authorized users.
6. **Documents:** DOCX export history, template version, creation time/user, Telegram outcome where reported, and download button. Explain that PDF conversion remains in Word/Docs.
7. **Activity:** authorized audit timeline.

The mobile survey form can reuse the existing section order. Avoid requiring the dashboard to faithfully render/edit the canvas drawing until a read-only renderer is working; preserve full data in the meantime.

10. DOCX and Telegram requirements

DOCX

- `template.docx` remains the company-approved source of truth. It is created/updated by the existing `tools/make_template.py` process from the client document; template changes need a version string and regression

export check.

- Client export builds the same flat docxtemplater data structure as today, including RF loops, photo loop, and the rail-all loop containing only measured labels.
- The app registers/export-syncs DOCX metadata and optionally uploads the generated DOCX so the dashboard can download the exact delivered file later.
- Do not generate a parallel PDF. The delivered DOCX is converted to PDF in Word/Google Docs/WPS when needed, as confirmed by the original app.

Telegram

- Preserve Android native behavior: silent attempt on export plus manual Send to Telegram action.
- UI must surface a non-sensitive success/failure state and permit a manual retry. Failure to send Telegram must not fail local DOCX export.
- Telegram token/chat configuration remains secret-injected into Android build/runtime configuration, not in source, API responses, or the web bundle.
- **Proposed later alternative:** server-side Telegram send, only after a credential owner, recipient policy, auditing, and retry/retention policy are approved.

11. Validation rules

Implement validation both in the client (fast feedback) and FastAPI (authoritative).

- Required before submitted: document reference (or server-generated survey number according to business policy), date, Site Name, Client if currently required by template, elevator ID where applicable, Prepared By, valid checklist choices, and field sections required by the existing form/template. Do not invent mandatory values; confirm exact required fields from current UI/template during implementation.
- Checklist accepts only Yes, No, N/A (normalizing legacy N-A only in import with audit note).
- Pinout validates four ordered objects with string use and color; allow blank entries only if the existing form allows them.
- RF metric validation: numeric RSRP/RSRQ/SINR/DL/UL/Ping where supplied; Band remains string/number-compatible as the existing app may capture either. Empty means unmeasured, not zero.
- floor_count is a positive integer within a configured sensible range; changing it must not silently delete existing RF readings. Prompt before removing out-of-range visible rows.
- Railing values are positive decimal millimetres when supplied; labels must match the drawing/export label convention.
- URLs use allowlisted http/https; filenames are display-only and sanitized.
- Server derives feasibility from valid filled cells and stores the calculation result/version. A manual override requires the actor and optional reason.

12. Local development and laptop test deployment

Development setup

- Use a monorepo and .env.example; never commit .env, Atlas URI, JWT keys, Telegram token/chat ID, or production file paths.
- Run FastAPI locally with a development CORS allowlist for the web dev host and Capacitor origin. Production/laptop deployment uses a strict explicit origin allowlist.
- MongoDB Atlas connection uses a least-privileged application user and TLS URI. Configure Atlas network access narrowly; for first test, allow only the laptop's current public IP, then tighten further.

- Use a local data directory configured by `FILE_STORAGE_ROOT`, e.g. a directory outside the repository and static asset tree. Back it up together with the database exports.
- Use uv or a locked Python dependency workflow; Node dependencies use a lockfile. Pin template and mobile build tooling versions.

Laptop deployment (MVP/test)

1. Build frontend static assets and run FastAPI as a managed local service/container.
2. Place a TLS-capable reverse proxy in front of it if phones access it over LAN/internet. Do not expose Uvicorn development mode directly to the public internet.
3. Configure `PUBLIC_BASE_URL`, Atlas URI, JWT/secret keys, file-storage root, allowed origins, and backup location in environment variables.
4. Provide a documented start/stop/status command and a health endpoint: `GET /healthz` (process alive) and `GET /readyz` (Atlas/storage reachable).
5. For access outside home/office LAN, use an approved secure VPN/reverse-tunnel approach; keep application authentication enabled. This is a test deployment, not a permanent public server.

13. Migration to cloud (proposed, after validated MVP)

1. Containerize FastAPI and move it to a managed container/app service.
2. Serve the web app from managed static hosting/CDN.
3. Swap `LocalFileStorage` for private S3-compatible object storage; retain all existing `storage_key` values or copy through a one-time migration ledger.
4. Keep MongoDB Atlas, move to a paid tier when capacity/backup needs justify it, and configure production IP/private networking as available.
5. Put secrets in a secret manager, configure managed TLS/domain, monitoring, alerting, and a tested restore process.
6. Use a migration checklist: backup, dry-run file count/checksum verification, read-only window if necessary, cutover, smoke test, rollback decision point. No schema rewrite is required.

14. Security, privacy, and backups

Security baseline

- TLS in transit; Atlas and object storage encryption at rest.
- Strong password hashing, short-lived access tokens, refresh-token rotation/revocation, role checks server-side, rate limiting, safe CORS, secure headers, and validation/size limits.
- Redact secrets and tokens from logs; structured logs include request IDs, not image bytes or full survey payloads by default.
- File authorization is checked on every access; file storage is private by default. Validate actual file content/type after upload.
- Use dependency scanning and lockfiles. Keep Telegram credentials out of version control and browser code.

Backups

MVP/test: scheduled MongoDB export/snapshot appropriate to Atlas tier plus a separate daily copy of the private file storage directory and report template. Keep at least one periodic restore-tested backup. Record backup time, survey count, file count, and result.

Production: managed database backups/point-in-time recovery according to selected Atlas tier, object-storage versioning/lifecycle policy, encrypted off-site backup copies, monitoring/alerts, and a documented restore runbook. Backups are not considered complete until restore testing succeeds.

15. Testing and quality gates

Automated tests

- Unit: Pydantic validation, legacy `svs_draft_v3` mapper, RF parser/normalizer, feasibility thresholds, manual override preservation, storage-key generation, authorization rules.
- API/integration: login/session rotation, role matrix, CRUD, pagination/filtering, optimistic concurrency (409), idempotent sync retry, photo upload/download authorization, audit events, report registration.
- Frontend: local-save state, queued offline change, reconnect sync, conflict UX, filters/URL state, dashboard table/detail rendering.
- DOCX regression: fixture draft -> generated DOCX; inspect required placeholder/loop completion and manually compare a representative rendering to approved template after each template change.
- Android manual/device tests: camera, photo resizing, cell-signal permissions/reads, dual-SIM carrier selection where test hardware exists, offline export, NativeFile open, Telegram send/retry. Web test confirms native features are correctly unavailable/fallback to typed fields/download.
- Security: dependency audit, unauthorized object access, invalid MIME/oversize files, injection/validation tests, expired/revoked token behavior.

MVP acceptance criteria

1. An authenticated engineer can create a survey on Android or web, edit all confirmed Site Visit Sheet sections, and recover local work after app/browser restart.
2. A legacy `svs_draft_v3` JSON imports into the new model with header, checklist, all three RF cell maps, drawings/dimensions, pinout, and photo metadata preserved; any unmapped field is reported, not discarded silently.
3. Offline changes queue and later sync. A conflicting same-field edit results in a visible conflict state rather than silent overwrite.
4. Photos are resized according to the test profile, stored outside MongoDB, shown in the survey detail, and inaccessible to an unauthorized user.
5. Dashboard searches and filters surveys by the specified fields, shows filtered statistics, lists results server-side, and opens all required detail tabs.
6. Existing DOCX export remains usable offline and preserves the approved template structure; web downloads and Android saves/opens it.
7. Android Telegram behavior remains available with safe error reporting; unsuccessful Telegram delivery does not prevent DOCX export.
8. Admin/manager/engineer/viewer authorization is tested; audit events exist for survey changes, status transitions, file operations, and report registration.
9. The laptop deployment has documented backup/restore steps, `/healthz` and `/readyz`, and a successful backup restore drill.

16. Phased implementation plan

Phase 0 - discovery and safety net

- Inventory current source, `template.docx`, `tools/make_template.py`, Android plugins, and actual form-required fields.

- Capture representative draft fixtures (empty, complete, multiple floors/photos/drawings, manual feasibility override) without exposing secrets.
- Define Pydantic schemas, API error format, role policy, and import mapping tests before UI refactor.

Phase 1 - preserve and modularize the existing app (MVP foundation)

- Split the monolithic HTML into maintainable frontend modules without changing the form's terminology/calculation/export behavior.
- Introduce a shared canonical survey model and versioned local draft store; add a tested `svs_draft_v3` importer.
- Keep Capacitor/native plugins and current offline DOCX path working.

Phase 2 - FastAPI, Atlas, files, and auth (MVP)

- Implement auth, users, sites/elevators, survey CRUD, RF calculation, audit events, storage adapter, local file storage, and photo endpoints.
- Create Atlas indexes and seed/admin bootstrap procedure.
- Implement optimistic revisions and basic sync endpoints.

Phase 3 - dashboard and synchronized survey workflow (MVP)

- Build dashboard search/filter/statistics/table and survey detail tabs.
- Wire manual/automatic online sync states, photo queue/retry, and conflict resolution UI.
- Register client-generated DOCX exports and make authorized DOCX download work in dashboard.

Phase 4 - laptop pilot and hardening (MVP exit)

- Deploy to laptop with TLS/access plan; run real field tests with selected engineers.
- Execute test suite, Android hardware checks, permission checks, backup/restore drill, and security review.
- Fix pilot issues before adding production-only scope.

Phase 5 - cloud migration and production hardening (proposed)

- Move files to object storage and API/web to managed hosting, maintain the storage adapter contract.
- Add managed secrets, monitoring/alerts, stronger backup policy, and deployment/rollback runbook.
- Evaluate SSO, tenant isolation, push/background sync, direct-to-object-storage uploads, and server-side Telegram only when business policy warrants them.

17. Recommended repository structure

```
elevator-rf-survey-app/
  README.md
  .env.example
  docker-compose.dev.yml           # proposed convenience only
  docs/
    BUILD_SPEC.md                 # this specification
    api-contract.md
    data-migration.md
    operations-runbook.md
  backend/
    pyproject.toml
    app/
      main.py
      api/v1/
      core/                       # config, security, logging, dependencies
      domain/                     # Pydantic models, RF calculations, legacy mapper
      services/                   # survey, sync, report, audit, authorization
```

```

repositories/          # MongoDB access only
storage/               # interface, local, s3 implementation
db/                   # client, indexes, seed/migrations
tests/
frontend/
package.json
src/
  app/                # routes/layout/auth
  features/survey/     # existing form sections, DOCX collection/export
  features/dashboard/
  features/sync/
  shared/              # API client, validation, UI, canonical models
  native/             # safe Capacitor bridge wrappers
public/
  template.docx
vendor/               # only if legacy bundled libraries retained
mobile/
  capacitor.config.*
  android-src/        # existing native plugin source of truth
  scripts/patch-android.*
tools/
  make_template.py    # existing template preparation workflow
  import_legacy_draft.py # controlled/tested import helper
infra/
  laptop/             # reverse proxy/service templates and runbook
  cloud/              # proposed deployment definitions
.github/workflows/
ci.yml
android.yml           # preserve/evolve existing APK build

```

Do not duplicate source-of-truth frontend files between `www/` and root as a long-term pattern. During transition, keep the current build working, then make a single build step produce Android web assets and static web output.

18. Implementation guardrails for Claude Code

1. Start with Phase 0 and tests/fixtures. Do not replace working DOCX or Android plugins until equivalent behavior is proven.
2. Treat the source report as authoritative for existing behavior. When code/template contradicts this brief, document the discrepancy and preserve working code until a product decision is made.
3. Every field removed/renamed from `svs_draft_v3` needs an explicit migration rule and test. Preserve unknown fields in `Legacy.unmapped` during import until reviewed.
4. Keep the MVP narrow: Atlas Free + laptop FastAPI + local private file storage + manual/reliable sync + dashboard. Label any background services, S3, SSO, web Telegram, notifications, or analytics as proposed unless approved.
5. Avoid embedding secrets, image Base64 payloads, or arbitrary MongoDB query fragments in the client/API.
6. Provide a concise operator README: setup, environment variables, laptop start/stop, initial admin creation, backup, restore, update, and troubleshooting.